



Universidad Nacional Autónoma de México
Facultad de Ingeniería

Sistemas Distribuidos — Semestre 2027-1

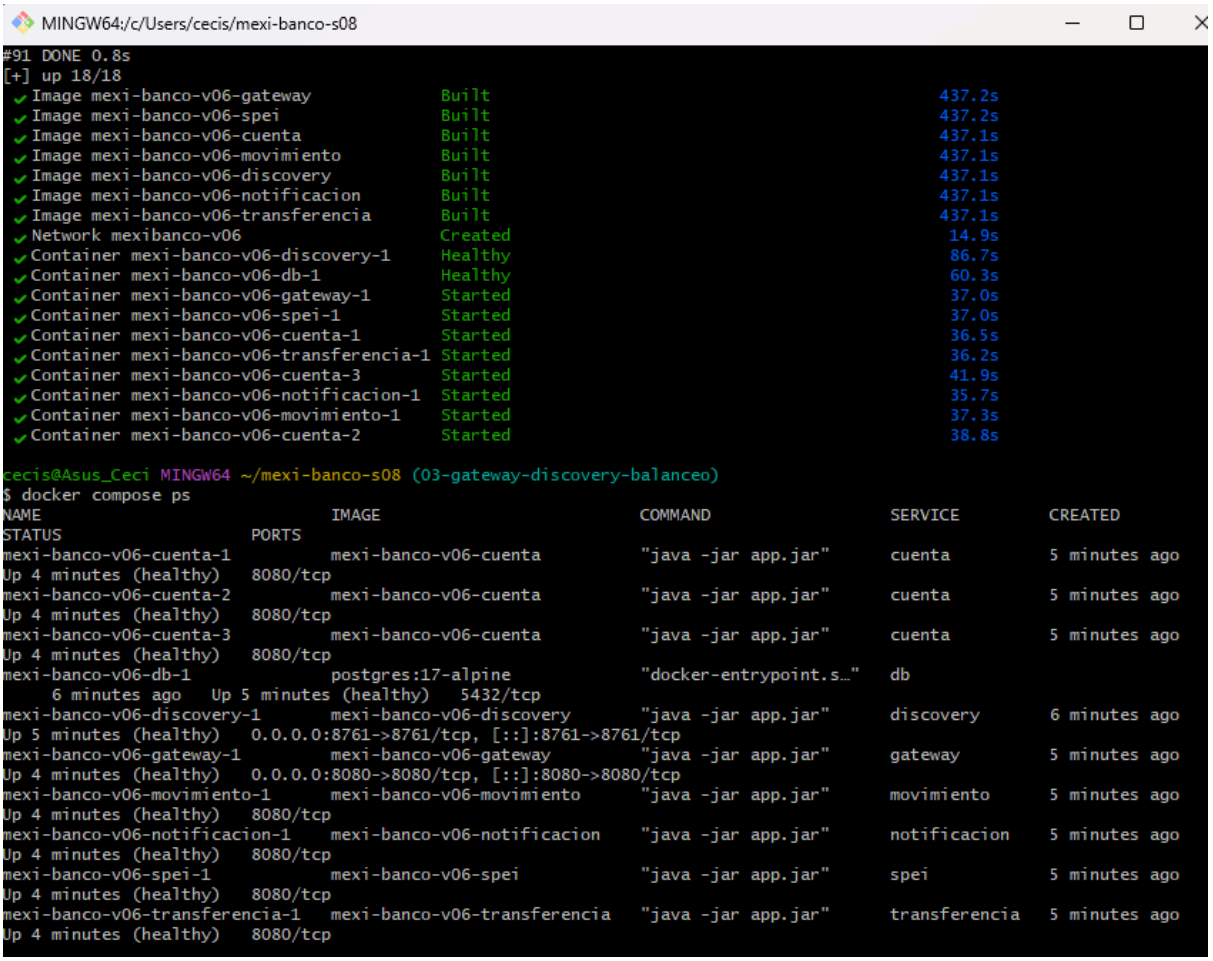
Laboratorio S08: Que la transferencia no deje dinero en el aire

Alumna: Cecilia Ximena Solís Cisneros
Fecha de entrega: 23 de octubre de 2026

Preparación del entorno S08

Descargué la rama 03-gateway-discovery-balanceo en la carpeta mexi-banco-s08. Durante la construcción apareció el error ./mvnw: not found. Ajusté los finales de línea de los archivos mvnw de Windows a formato LF y volví a construir.

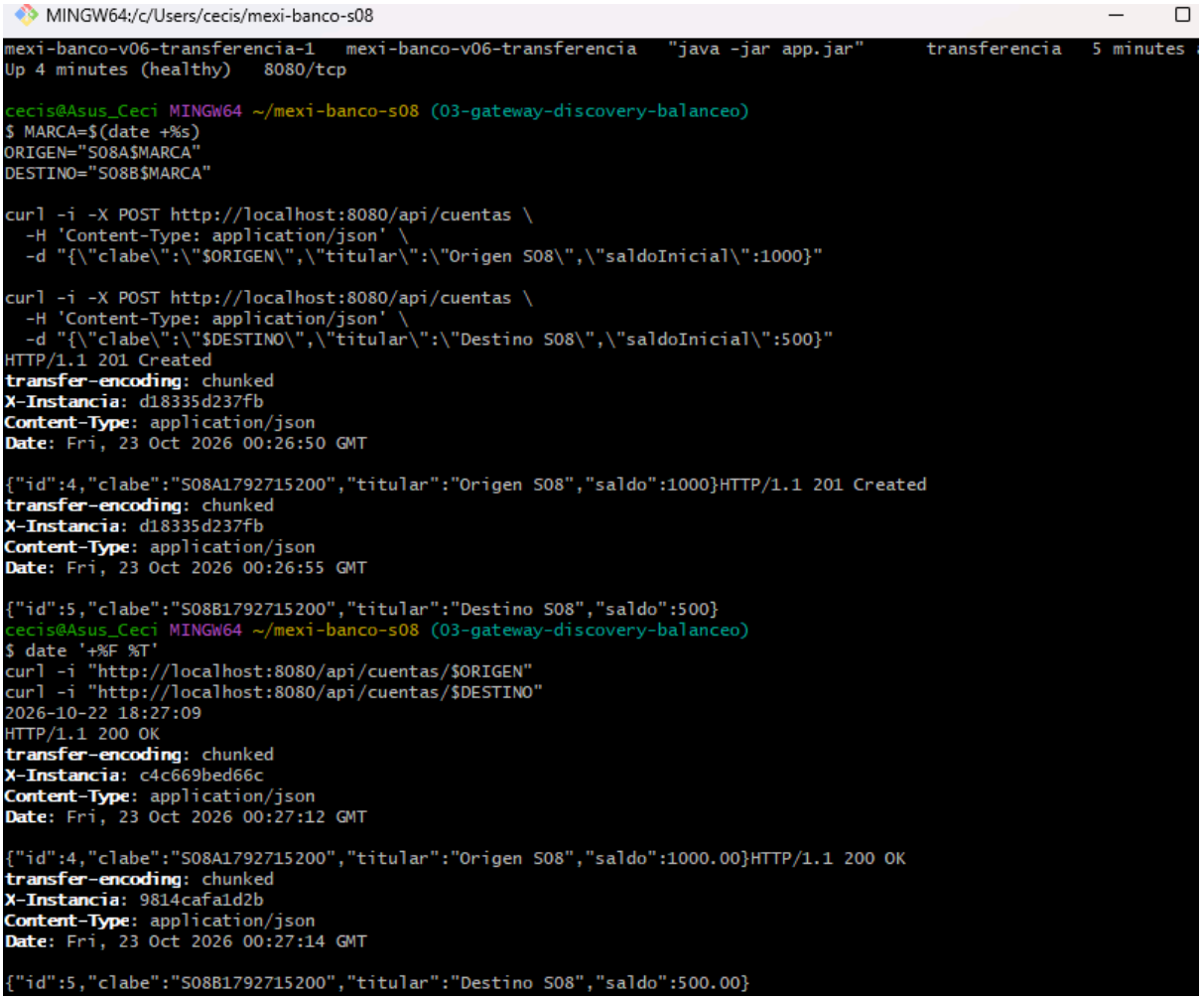
Después comprobé con docker compose ps que los diez contenedores estaban healthy, incluidas las tres copias de cuenta.



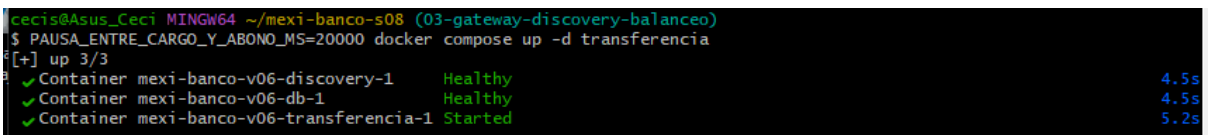
Fecha y hora de la comprobación: 6:20, 22/19/2026.

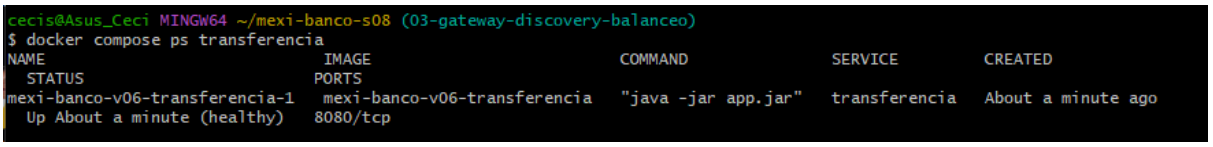
Parte 0 El problema antes de implementar la saga

El 22 de octubre de 2026, a las 18:27, consulté dos cuentas mediante el gateway. La cuenta de origen S08A1792715200 tenía saldo 1000.00 y la cuenta de destino S08B1792715200 tenía saldo 500.00. Ambas consultas respondieron 200 OK. El total inicial era 1500.00:



Configuré transferencia con PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 para introducir una pausa de 20 segundos entre el cargo y el abono. Después comprobé que el contenedor de transferencia estaba healthy. Esta pausa permite detener cuenta después del cargo y antes de que se intente el abono.





A las 18:41:07 consulté nuevamente las cuentas. El origen tenía 800.00 y el destino 700.00, por lo que la transferencia de 200 sí se realizó, aunque el gateway respondió con timeout. Esto demuestra que un timeout no significa necesariamente que la operación no se ejecutó. En este intento no reproduje la falla del abono, porque no detuve cuenta durante la pausa.

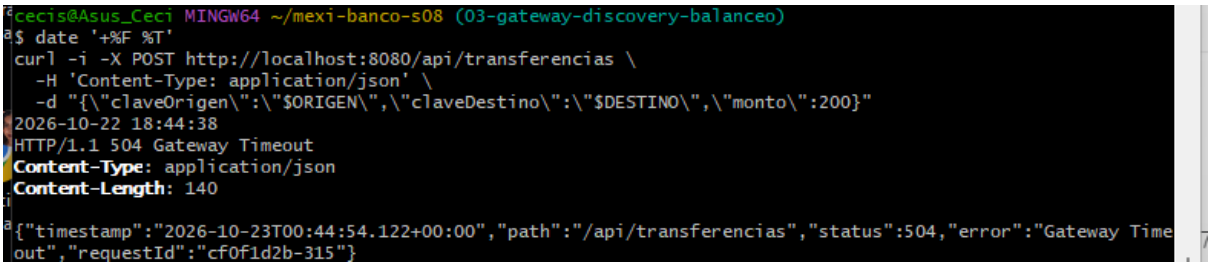


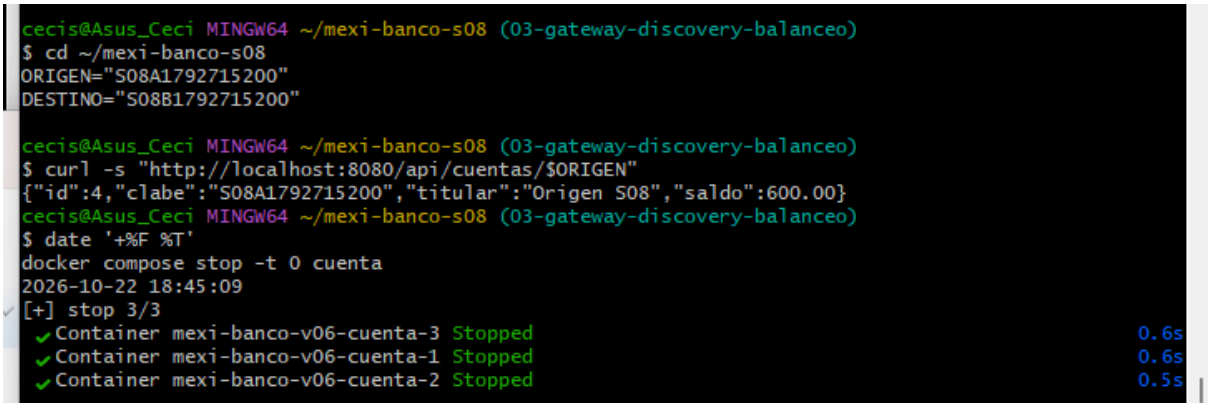
Segundo intento de reproducción de la falla

Antes de este intento, origen tenía 800.00 y destino 700.00. El 22 de octubre de 2026, a las 18:44:38, solicité una nueva transferencia de 200. El gateway respondió 504 Gateway Timeout aproximadamente a las 18:44:54.

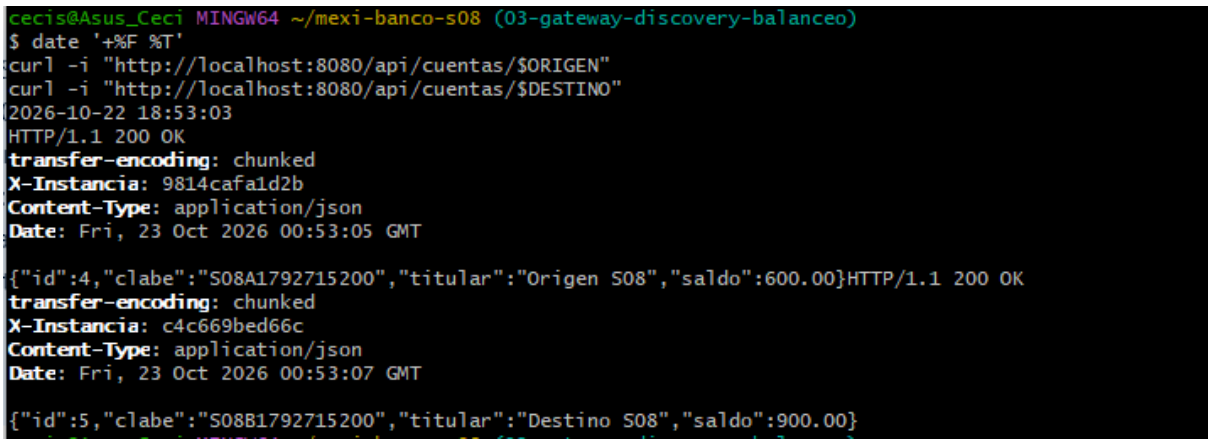
En la segunda terminal consulté origen y observé un saldo de 600.00, lo que confirma que el cargo se había aplicado. Después detuve las tres copias de cuenta a las 18:45:09.

El apagado ocurrió 31 segundos después de iniciar la solicitud. Como la pausa configurada era de 20 segundos, todavía no puedo asegurar que haya impedido el abono. Para comprobarlo, consultaré ambos saldos después de recuperar cuenta.





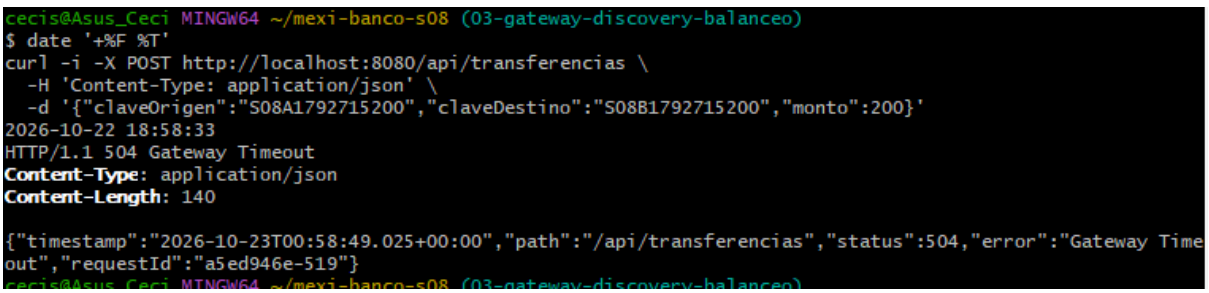
A las 18:53:03 consulté los saldos después de recuperar cuenta. Origen tenía 600.00 y destino 900.00; el total seguía siendo 1500.00. Esto confirma que la segunda transferencia también se completó, aunque el gateway respondió 504. El apagado ocurrió demasiado tarde para impedir el abono:



Tercer intento con apagado durante la pausa

El 22 de octubre de 2026, a las 18:58:33, inicié una transferencia de 200. Antes del intento, origen tenía 600.00 y destino 900.00.

Preparé una consulta automática que detuvo las tres copias de cuenta al detectar que el saldo de origen había bajado a 400.00. El apagado se activó a las 18:58:35, durante la pausa de 20 segundos entre el cargo y el abono. El gateway respondió 504 aproximadamente a las 18:58:49. Quedó pendiente consultar los saldos finales para comprobar si el abono falló:



```
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ ORIGEN="S08A1792715200"

while true; do
  RESPUESTA=$(curl -s --max-time 3 "http://localhost:8080/api/cuentas/$ORIGEN")
  if printf '%s' "$RESPUESTA" | grep -Eq '"saldo"[[:space:]]*:[[:space:]]*400([.,}]|$)'; then
    date '+%F %T'
    printf '%s\n' "$RESPUESTA"
    docker compose stop -t 0 cuenta
    break
  fi
  sleep 1
done
2026-10-22 18:58:35
{"id":4,"clabe":"S08A1792715200","titular":"Origen S08","saldo":400.00}
[+] stop 3/3
 ✔ Container mexi-banco-v06-cuenta-3 Stopped                                    1.
 ✔ Container mexi-banco-v06-cuenta-1 Stopped                                    1.
 ✔ Container mexi-banco-v06-cuenta-2 Stopped                                    1.
```

A las 18:58:54, los logs de transferencia registraron “No instances available for cuenta” al ejecutar CuentaCliente.abonar. Esto confirma que el intento de abono falló porque las tres copias de cuenta estaban detenidas. El cargo ya se había aplicado; falta comprobar los saldos después de recuperar el servicio:

```
MINGW64:/c/Users/cecis/mexi-banco-s08

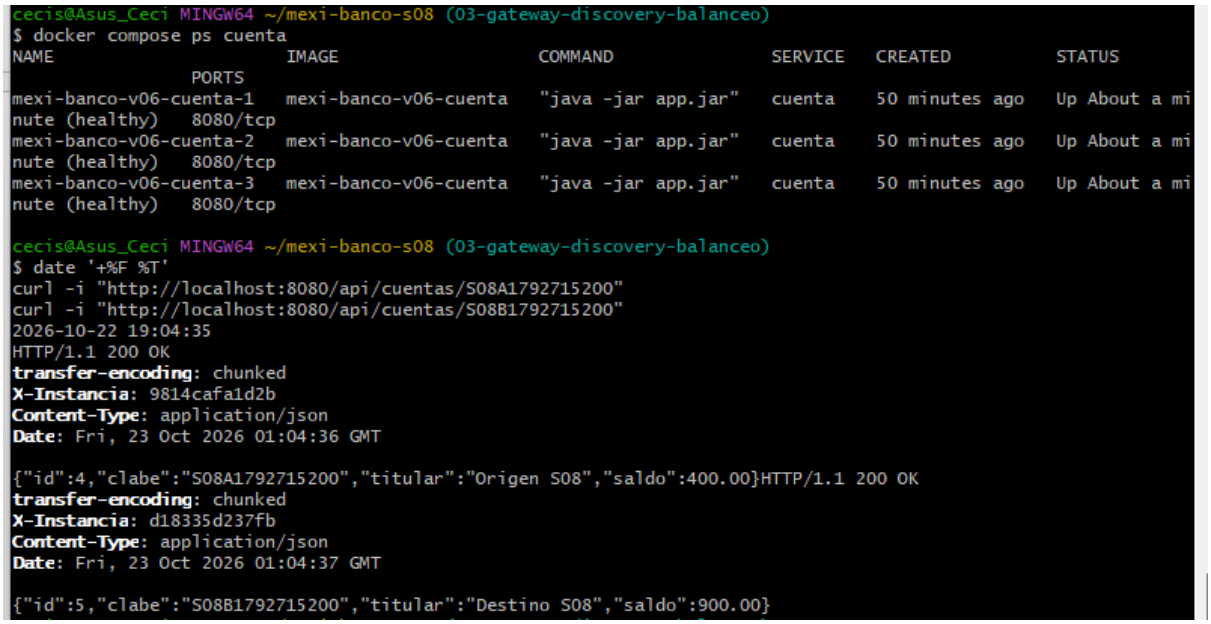
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ docker compose logs --since=5m --tail=100 transferencia
transferencia-1  | 2026-10-23T00:58:54.343Z  WARN 1 --- [transferencia] [nio-8080-exec-6] o.s.c.l.core.RoundRobi
nLoadBalancer      : No servers available for service: cuenta
transferencia-1  | 2026-10-23T00:58:54.349Z ERROR 1 --- [transferencia] [nio-8080-exec-6] o.a.c.c.C.[.[.[/].[dis
patcherServlet]    : Servlet.service() for servlet [dispatcherServlet] in context with path [] threw exception [
Request processing failed: java.lang.IllegalStateException: No instances available for cuenta] with root cause
transferencia-1  |
transferencia-1  | java.lang.IllegalStateException: No instances available for cuenta
transferencia-1  |      at org.springframework.cloud.loadbalancer.blocking.client.BlockingLoadBalancerClient.exe
cute(BlockingLoadBalancerClient.java:78) ~[spring-cloud-loadbalancer-4.1.4.jar!/:4.1.4]
transferencia-1  |      at org.springframework.cloud.client.loadbalancer.LoadBalancerInterceptor.intercept(LoadB
alancerInterceptor.java:55) ~[spring-cloud-commons-4.1.4.jar!/:4.1.4]
transferencia-1  |      at org.springframework.cloud.client.loadbalancer.DeferringLoadBalancerInterceptor.interc
ept(DeferringLoadBalancerInterceptor.java:51) ~[spring-cloud-commons-4.1.4.jar!/:4.1.4]
transferencia-1  |      at org.springframework.http.client.InterceptingClientHttpRequest$InterceptingRequestExec
ution.execute(InterceptingClientHttpRequest.java:88) ~[spring-web-6.1.13.jar!/:6.1.13]
transferencia-1  |      at org.springframework.http.client.InterceptingClientHttpRequest.executeInternal(Interce
ptingClientHttpRequest.java:72) ~[spring-web-6.1.13.jar!/:6.1.13]
transferencia-1  |      at org.springframework.http.client.AbstractBufferingClientHttpRequest.executeInternal(Ab
stractBufferingClientHttpRequest.java:48) ~[spring-web-6.1.13.jar!/:6.1.13]
transferencia-1  |      at org.springframework.http.client.AbstractClientHttpRequest.execute(AbstractClientHttpR
equest.java:66) ~[spring-web-6.1.13.jar!/:6.1.13]
transferencia-1  |      at org.springframework.web.client.DefaultRestClient$DefaultRequestBodyUriSpec.exchangeIn
ternal(DefaultRestClient.java:492) ~[spring-web-6.1.13.jar!/:6.1.13]
transferencia-1  |      at org.springframework.web.client.DefaultRestClient$DefaultRequestBodyUriSpec.retrieve(D
efaultRestClient.java:458) ~[spring-web-6.1.13.jar!/:6.1.13]
transferencia-1  |      at mx.mexibanco.clientes.CuentaCliente.abonar(CuentaCliente.java:36) ~[!/:0.6.0]
transferencia-1  |      at mx.mexibanco.transferencia.TransferenciaService.transferir(TransferenciaService.java:
57) ~[!/:0.6.0]
transferencia-1  |      at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(Unknown Source) ~[na
:na]
transferencia-1  |      at java.base/java.lang.reflect.Method.invoke(Unknown Source) ~[na:na]
transferencia-1  |      at org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java
:355) ~[spring-aop-6.1.13.jar!/:6.1.13]
transferencia-1  |      at org.springframework.aop.framework.CglibAopProxy$DynamicAdvisedInterceptor.intercept(C
glibAopProxy.java:716) ~[spring-aop-6.1.13.jar!/:6.1.13]
transferencia-1  |      at mx.mexibanco.transferencia.TransferenciaService$$SpringCGLIB$$0.transferir(<generated
>) ~[!/:0.6.0]
transferencia-1  |      at mx.mexibanco.transferencia.TransferenciaController.transferir(TransferenciaController
.java:28) ~[!/:0.6.0]
```

Resultado del cargo sin abono

El 22 de octubre de 2026, a las 19:04:35, consulté las cuentas después de recuperar el servicio. Origen tenía 400.00 y destino conservaba 900.00.

Antes de este intento, los saldos eran 600.00 y 900.00, con un total de 1500.00. Después quedaron en 1300.00: los 200 salieron de origen, pero no llegaron a destino ni se devolvieron.

Esto ocurrió porque el cargo y el abono son llamadas HTTP independientes. El cargo ya estaba confirmado cuando falló el abono, y la versión actual no tiene una compensación que devuelva el dinero:

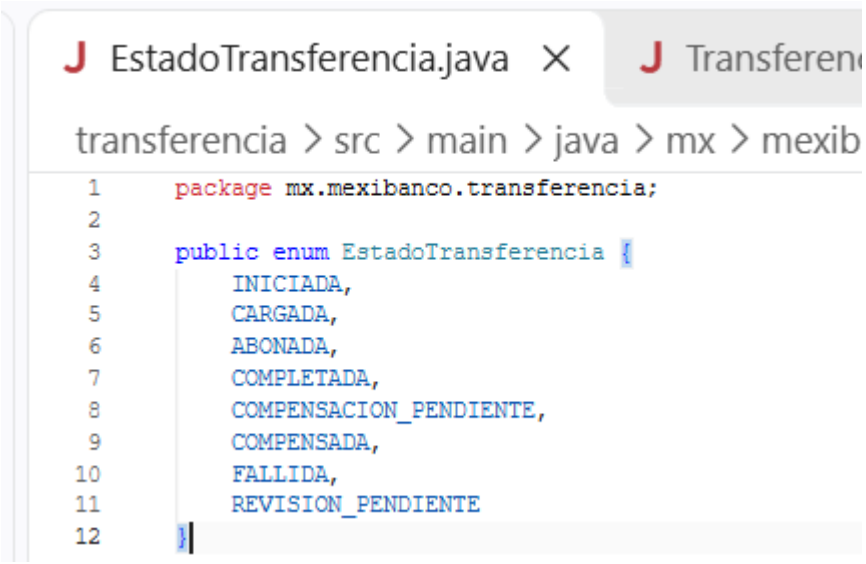


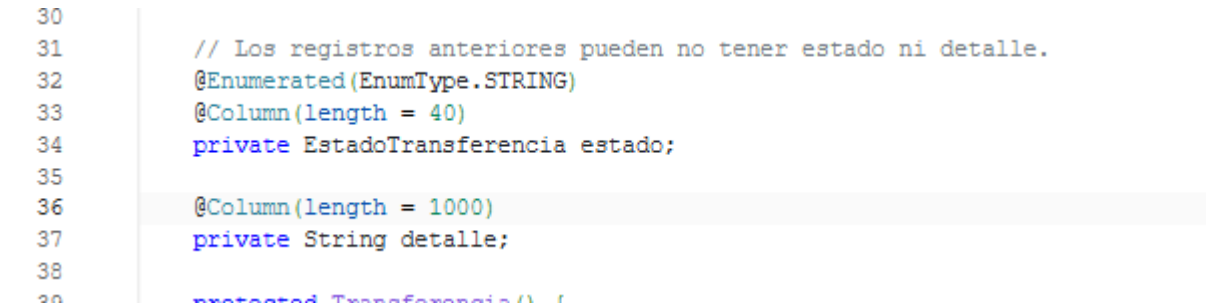
Estados de la transferencia

Agregué los campos estado y detalle a Transferencia.java. El estado permitirá identificar en qué paso quedó la operación, y el detalle explicará lo sucedido. Las transferencias nuevas se crean con el estado INICIADA.

También creé EstadoTransferencia.java con los estados INICIADA, CARGADA, ABONADA, COMPLETADA, COMPENSACION_PENDIENTE, COMPENSADA, FALLIDA y REVISION_PENDIENTE. Estos permiten distinguir una operación terminada, una devolución pendiente y un resultado que necesita revisión.

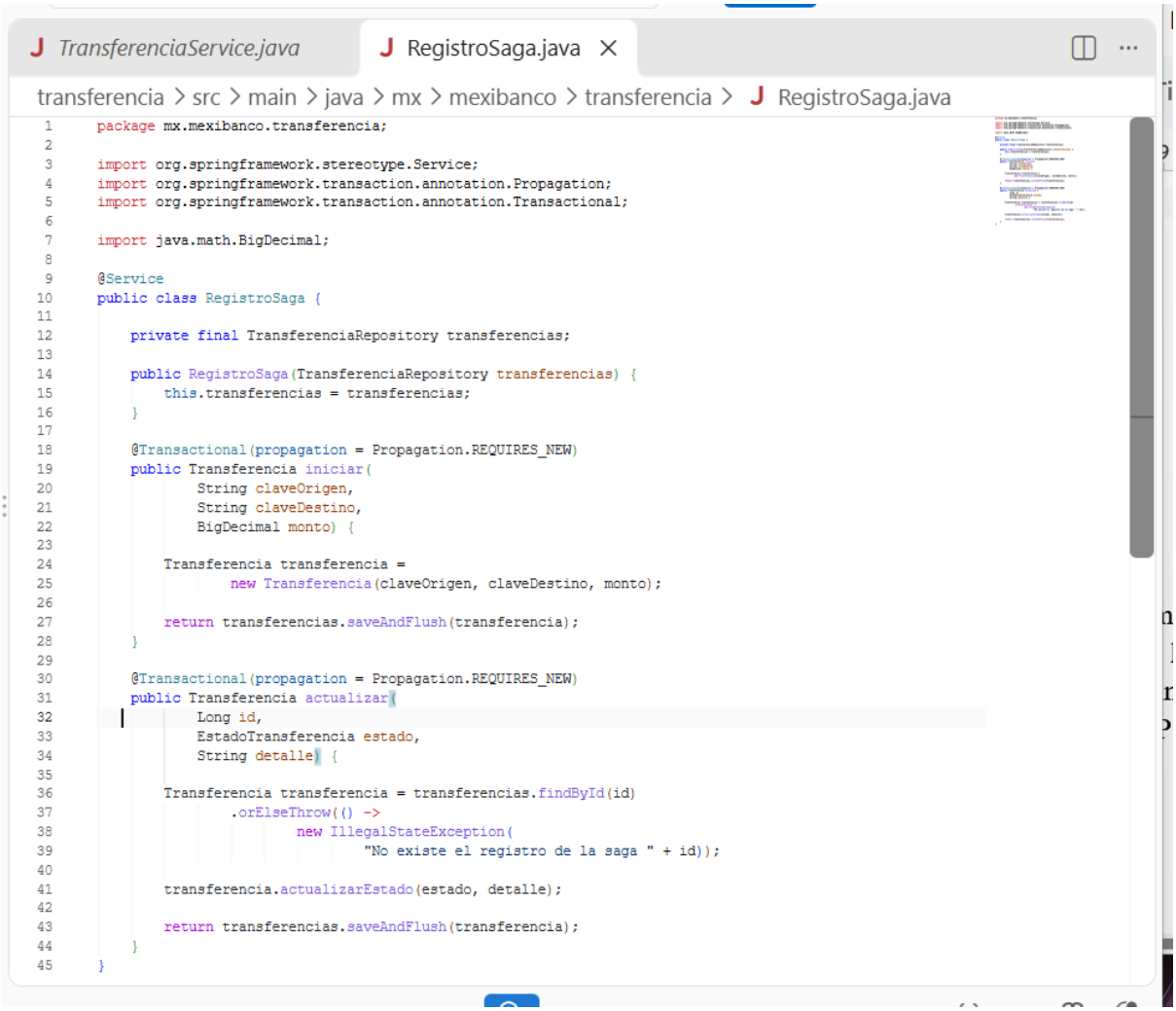
Por ahora solo agregué la estructura para guardar esta información. Todavía falta modificar el servicio para registrar los cambios de estado y ejecutar la compensación.





Registro de los pasos

Creé RegistroSaga para guardar el inicio y los cambios de estado en transacciones locales independientes. Así, una falla posterior no deshace los estados que ya se confirmaron en la base de transferencia. Esto permite consultar el último paso registrado, aunque no garantiza por sí solo conocer el resultado de una llamada HTTP cuya respuesta se perdió.



Falla controlada del abono

Agregué una falla de prueba configurable mediante ABONO_FALLA_CLABE. Cuando el destino coincide con esa CLABE, transferencia rechaza el abono antes de enviarlo a cuenta. Esto permitirá probar la compensación manteniendo cuenta disponible.

Separé el abono al destino del abono utilizado para devolver dinero al origen, para que la falla de prueba no impida la devolución. Todavía falta conectar este comportamiento al flujo de la saga:

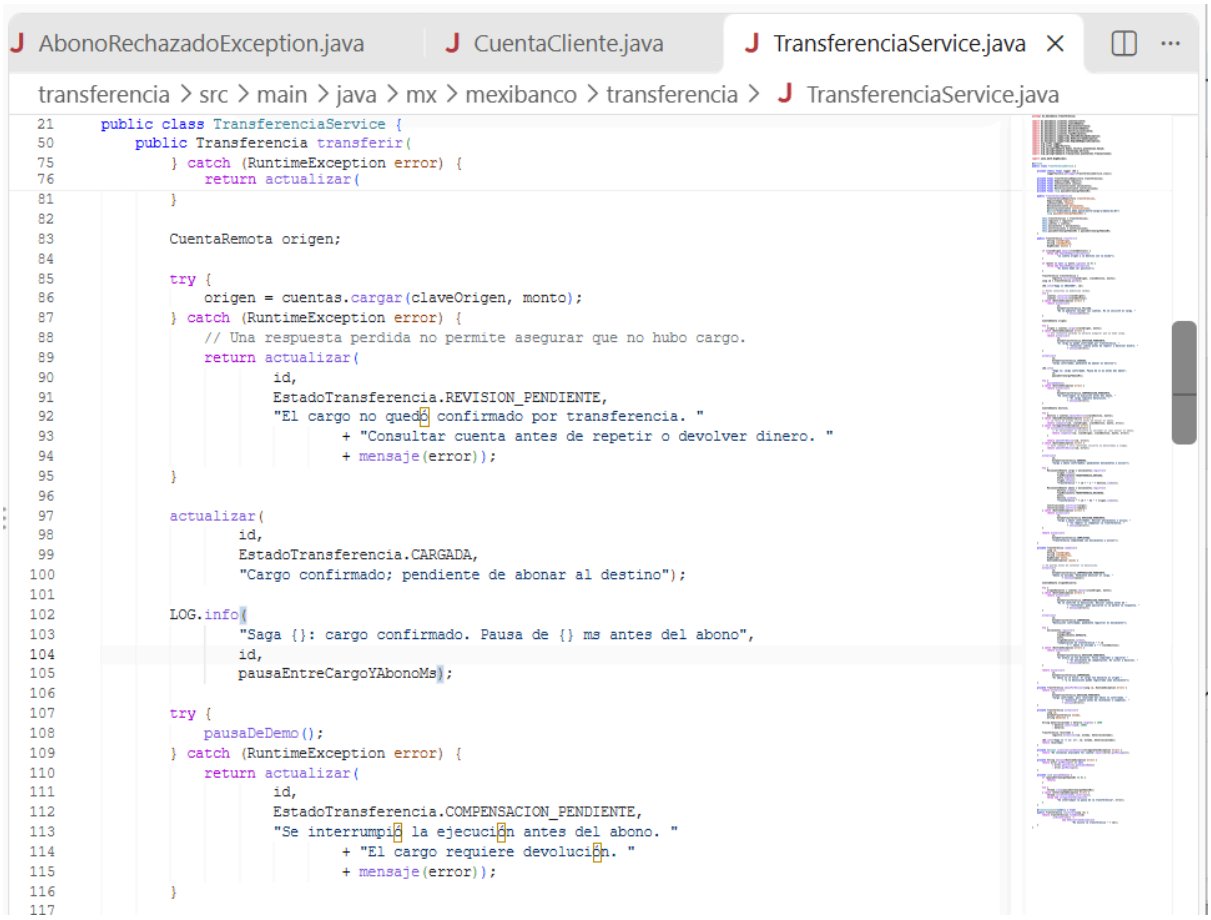


Orquestación y compensación

Modifiqué TransferenciaService para registrar la operación antes de mover el dinero y guardar los estados alcanzados. Si la falla de prueba impide enviar el abono, el servicio intenta devolver el monto al origen y registrar la devolución como un movimiento.

Antes de devolver, guarda COMPENSACION_PENDIENTE. Si la devolución se confirma y se registra su movimiento, termina en COMPENSADA. Si no puede confirmar la devolución, conserva el estado pendiente.

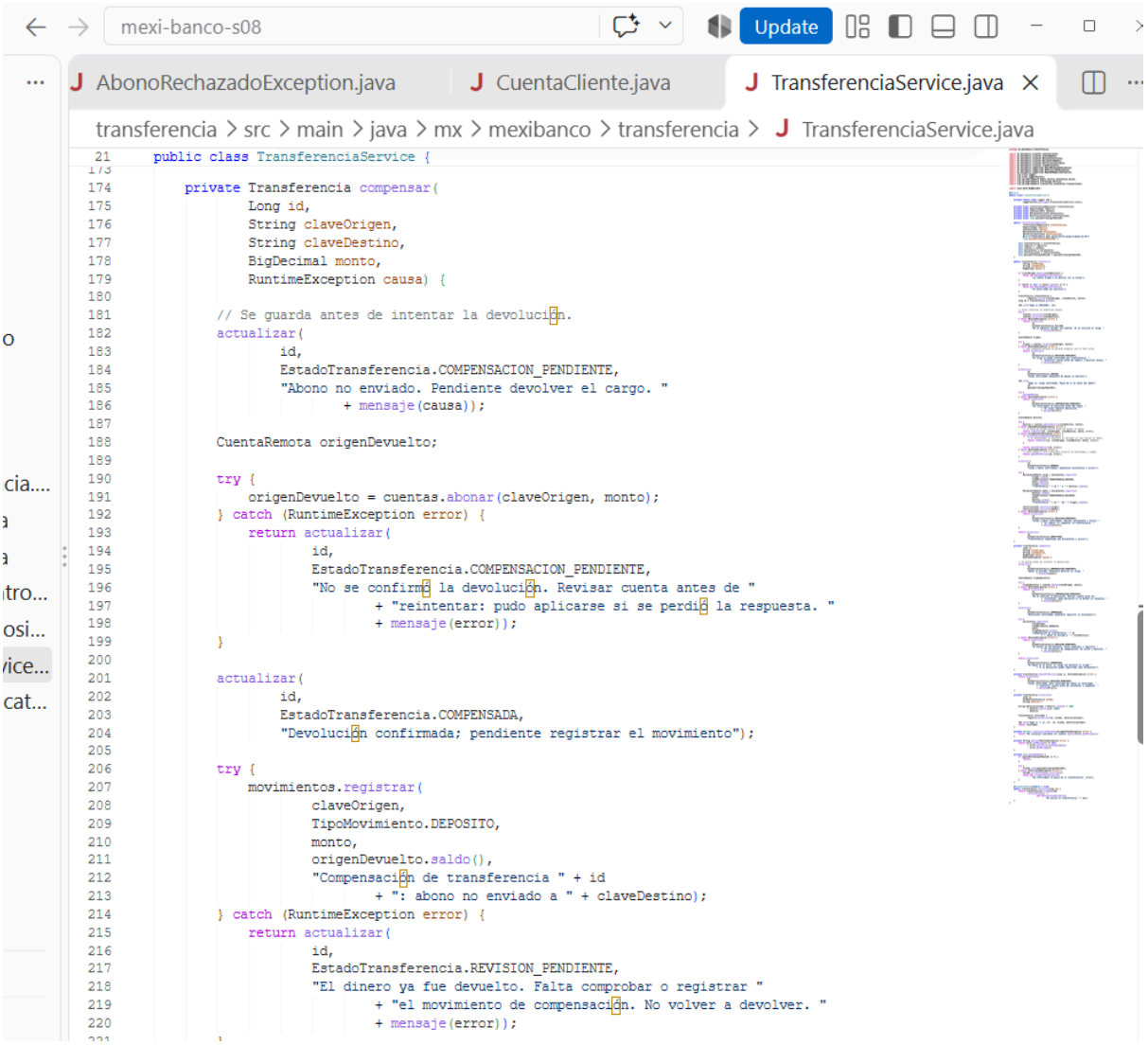
Cuando el resultado de una llamada es incierto, deja REVISION_PENDIENTE para evitar repetir o compensar a ciegas. Esta versión todavía no tiene recuperación automática ni reintentos idempotentes de compensación.


AbonoRechazadoException.java
CuentaCliente.java
TransferenciaService.java
transferencia > src > main > java > mx > mexibanco > transferencia > TransferenciaService.java
21 public class TransferenciaService {
50 public Transferencia transferir(
75 } catch (RuntimeException error) {
76 return actualizar(
81 }
82
83 CuentaRemota origen;
84
85 try {
86 origen = cuentas.cargar(claveOrigen, monto);
87 } catch (RuntimeException error) {
88 // Una respuesta perdida no permite asegurar que no hubo cargo.
89 return actualizar(
90 id,
91 EstadoTransferencia.REVISION_PENDIENTE,
92 "El cargo no quedó confirmado por transferencia. "
93 + "Consultar cuenta antes de repetir o devolver dinero. "
94 + mensaje(error));
95 }
96
97 actualizar(
98 id,
99 EstadoTransferencia.CARGADA,
100 "Cargo confirmado; pendiente de abonar al destino");
101
102 LOG.info(
103 "Saga {}: cargo confirmado. Pausa de {} ms antes del abono",
104 id,
105 pausaEntreCargoYAbonoMs);
106
107 try {
108 pausaDeDemo();
109 } catch (RuntimeException error) {
110 return actualizar(
111 id,
112 EstadoTransferencia.COMPENSACION_PENDIENTE,
113 "Se interrumpió la ejecución antes del abono. "
114 + "El cargo requiere devolución. "
115 + mensaje(error));
116 }
117

AbonoRechazadoException.java | CuentaCliente.java | TransferenciaService.java

transferencia > src > main > java > mx > mexibanco > transferencia > TransferenciaService.java

```
21  public class TransferenciaService {
50      public Transferencia transferir(
117
118         CuentaRemota destino;
119
120         try {
121             destino = cuentas.abonarDestino(claveDestino, monto);
122         } catch (AbonoRechazadoException error) {
123             // La falla de prueba ocurre antes de enviar el abono.
124             return compensar(id, claveOrigen, claveDestino, monto, error);
125         } catch (IllegalStateException error) {
126             if (sinInstanciasDeCuenta(error)) {
127                 // El balanceador no encontró un servidor al cual enviar el abono.
128                 return compensar(id, claveOrigen, claveDestino, monto, error);
129             }
130
131             return abonoPorRevisar(id, error);
132         } catch (RuntimeException error) {
133             // Ante timeout u otro resultado incierto no devolvemos a ciegas.
134             return abonoPorRevisar(id, error);
135         }
136
137         actualizar(
138                 id,
139                 EstadoTransferencia.ABONADA,
140                 "Cargo y abono confirmados; pendientes movimientos y avisos");
141
142         try {
143             MovimientoRemoto cargo = movimientos.registrar(
144                     origen.clabe(),
145                     TipoMovimiento.TRANSFERENCIA_ENVIADA,
146                     monto.negate(),
147                     origen.saldo(),
148                     "Transferencia " + id + " a " + destino.clabe());
149
150             MovimientoRemoto abono = movimientos.registrar(
151                     destino.clabe(),
152                     TipoMovimiento.TRANSFERENCIA_RECIBIDA,
153                     monto,
154                     destino.saldo(),
155                     "Transferencia " + id + " de " + origen.clabe());
156
157             notificaciones.notificar(cargo);
158             notificaciones.notificar(abono);
159         } catch (RuntimeException error) {
160             return actualizar(
161                     id,
162                     EstadoTransferencia.REVISION_PENDIENTE,
163                     "Cargo y abono confirmados. Revisar movimientos o avisos; "
164                             + "no repetir ni compensar la transferencia. "
```



Respuesta al cliente

Modifiqué el controlador para responder según el estado de la saga. La respuesta incluye el identificador, el estado y una explicación de lo ocurrido. Así se distingue una transferencia completada de una compensada o pendiente de revisión. Las operaciones pendientes todavía necesitan atención; esta versión no las recupera automáticamente.

TransferenciaController.java

transferencia > src > main > java > mx > mexibanco > transferencia > TransferenciaContro

```java
package mx.mexibanco.transferencia;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;

@RestController
@RequestMapping("/transferencias")
public class TransferenciaController {

    private final TransferenciaService transferencias;

    public TransferenciaController(TransferenciaService transferencias) {
        this.transferencias = transferencias;
    }

    public record SolicitudTransferencia(
            @NotBlank String claveOrigen,
            @NotBlank String claveDestino,
            @NotNull @Positive BigDecimal monto) {
    }

    @PostMapping
    public ResponseEntity<Transferencia> transferir(
            @Valid @RequestBody SolicitudTransferencia solicitud) {

        Transferencia resultado = transferencias.transferir(
                solicitud.claveOrigen(),
                solicitud.claveDestino(),
                solicitud.monto());

        HttpStatus respuesta = switch (resultado.getEstado()) {
            case COMPLETADA -> HttpStatus.CREATED;
            case COMPENSADA -> HttpStatus.CONFLICT;
            case FALLIDA -> HttpStatus.UNPROCESSABLE_ENTITY;
            default -> HttpStatus.ACCEPTED;
        };

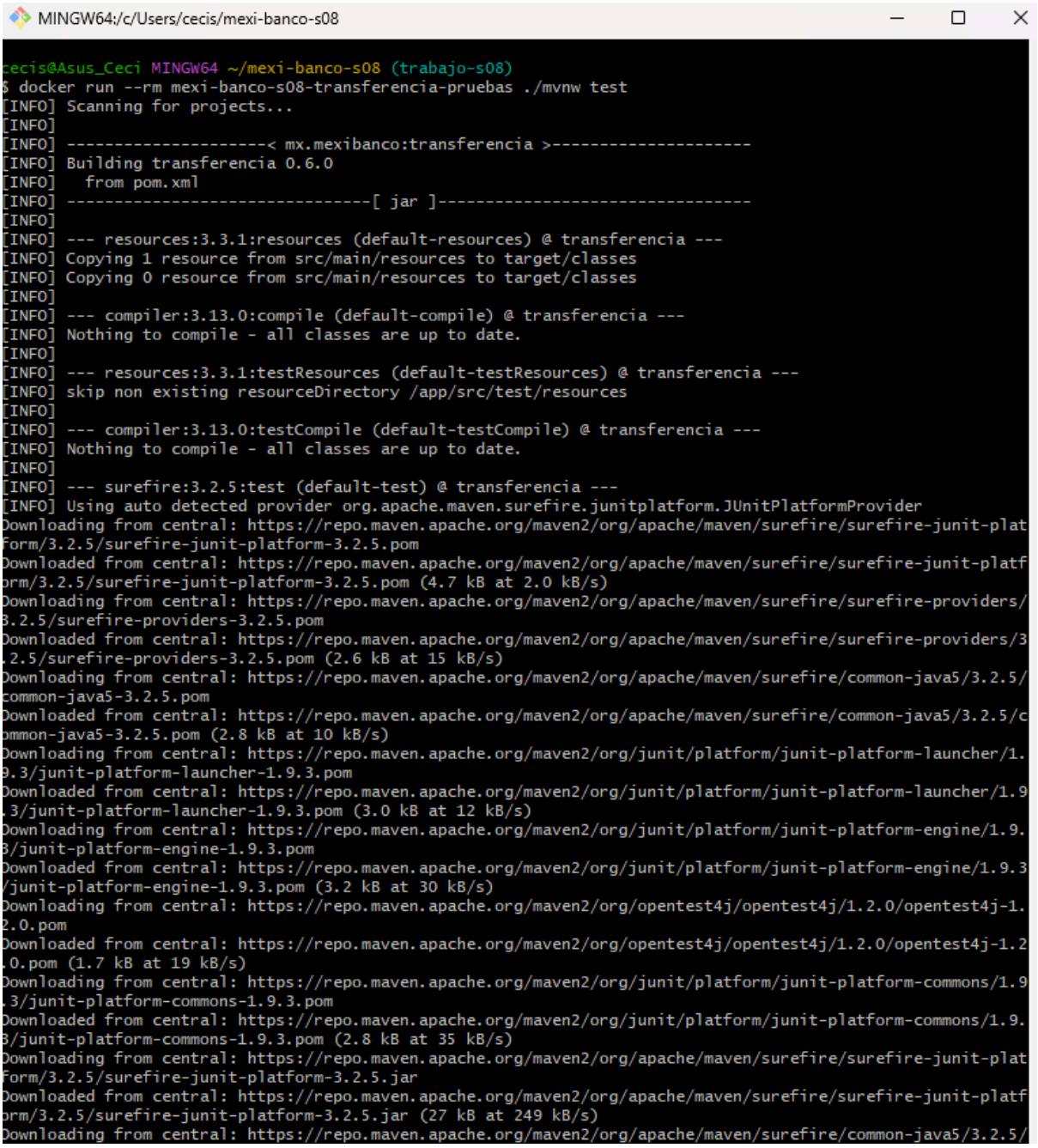
        return ResponseEntity.status(respuesta).body(resultado);
    }

    @GetMapping("/{id}")
    public Transferencia consultar(@PathVariable Long id) {
        return transferencias.consultar(id);
```

Pruebas automatizadas

El 22 de octubre de 2026, aproximadamente a las 20:23, ejecuté las pruebas de transferencia dentro de Docker. El resultado fue de 11 pruebas ejecutadas, sin fallos, errores ni pruebas omitidas.

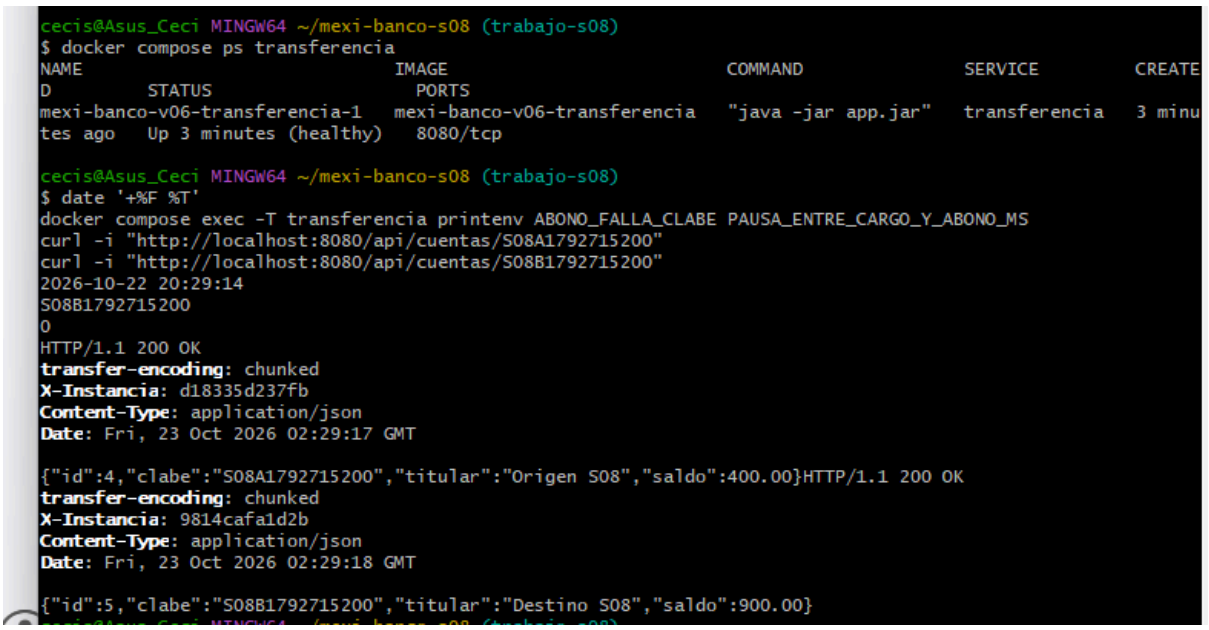
Las pruebas verifican, entre otros casos, la transferencia exitosa, la compensación ante una falla controlada y el estado pendiente cuando no se confirma la devolución. Estas comprobaciones utilizan servicios simulados; todavía falta comprobar los saldos y estados con los servicios reales.


MINGW64:/c/Users/cecis/mexi-banco-s08
cecis@Asus_Ceci MINGW64 ~/mexi-banco-s08 (trabajo-s08)
$ docker run --rm mexi-banco-s08-transferencia-pruebas ./mvnw test
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< mx.mexibanco:transferencia >----------------------
[INFO] Building transferencia 0.6.0
[INFO]   from pom.xml
[INFO] --------------------------------[jar]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ transferencia ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ transferencia ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ transferencia ---
[INFO] skip non existing resourceDirectory /app/src/test/resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ transferencia ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ transferencia ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit-plat
form/3.2.5/surefire-junit-platform-3.2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit-platf
orm/3.2.5/surefire-junit-platform-3.2.5.pom (4.7 kB at 2.0 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-providers/
3.2.5/surefire-providers-3.2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-providers/3
.2.5/surefire-providers-3.2.5.pom (2.6 kB at 15 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/
common-java5-3.2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/c
ommon-java5-3.2.5.pom (2.8 kB at 10 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.
9.3/junit-platform-launcher-1.9.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.9
.3/junit-platform-launcher-1.9.3.pom (3.0 kB at 12 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-engine/1.9.
3/junit-platform-engine-1.9.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-engine/1.9.3
/junit-platform-engine-1.9.3.pom (3.2 kB at 30 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/opentest4j/opentest4j/1.2.0/opentest4j-1.
2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/opentest4j/opentest4j/1.2.0/opentest4j-1.2
.0.pom (1.7 kB at 19 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-commons/1.9
.3/junit-platform-commons-1.9.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-commons/1.9.
3/junit-platform-commons-1.9.3.pom (2.8 kB at 35 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit-plat
form/3.2.5/surefire-junit-platform-3.2.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit-platf
orm/3.2.5/surefire-junit-platform-3.2.5.jar (27 kB at 249 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/



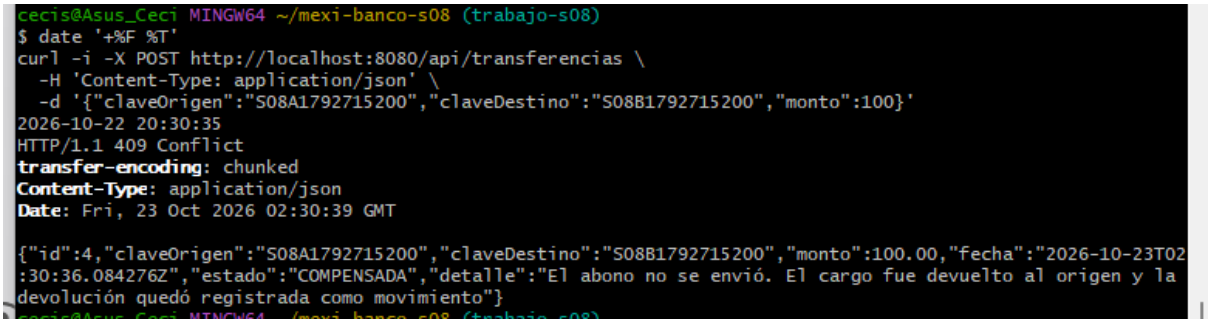
Preparación de la prueba de compensación

El 22 de octubre de 2026, a las 20:29:14, comprobé que ABONO_FALLA_CLABE estaba configurada con S08B1792715200 y que la pausa era de 0 milisegundos. Antes de la prueba, origen tenía 400.00 y destino 900.00. Estos son los saldos posteriores al experimento anterior; esta prueba no pretende reparar aquella operación.



Respuesta de la compensación

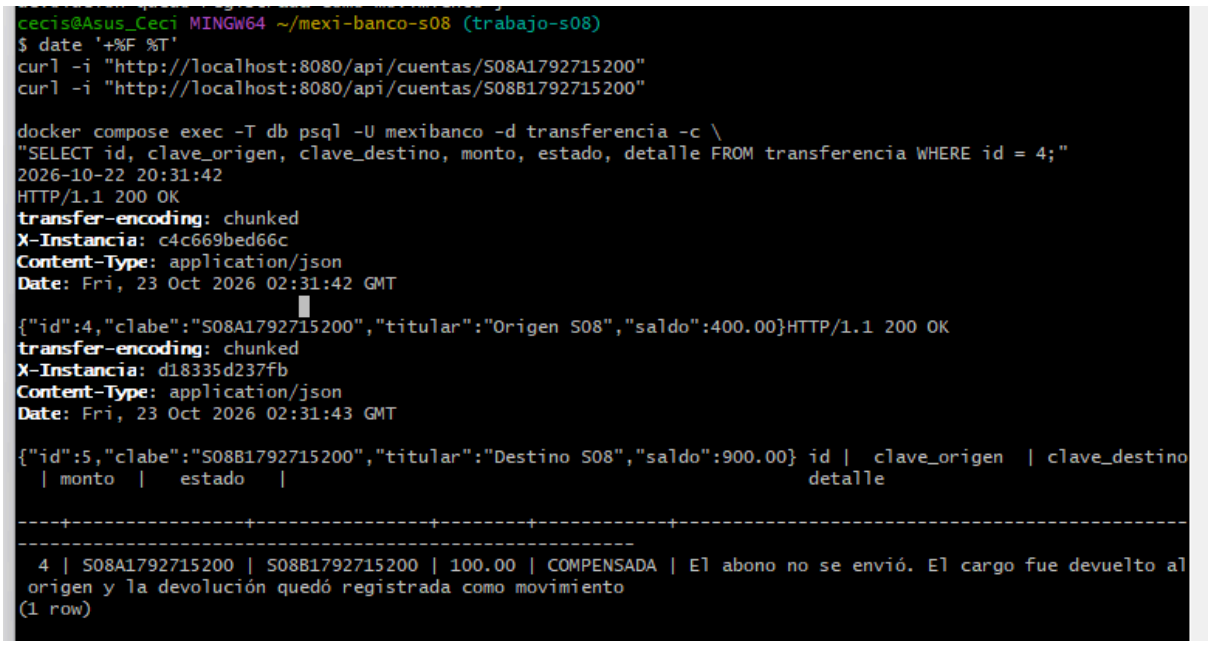
El 22 de octubre de 2026, a las 20:30:35, solicité una transferencia de 100. La falla configurada impidió enviar el abono al destino. La operación quedó registrada con ID 4 y respondió 409 Conflict, con estado COMPENSADA. El detalle informó que el cargo fue devuelto al origen y que la devolución quedó registrada como movimiento.



Comprobación de saldos y estado

A las 20:31:42 consulté ambas cuentas. Origen conservaba 400.00 y destino 900.00, los mismos saldos anteriores a esta prueba. Esto confirma que el cargo de 100 fue devuelto sin abonar al destino.

También consulté directamente la base de transferencia. La operación con ID 4 quedó guardada con monto 100.00 y estado COMPENSADA.



Movimiento de compensación

Consulté los movimientos de origen y encontré el registro 12, de tipo DEPOSITO, por 100.00 y con saldo resultante 400.00. Su referencia indica “Compensación de transferencia 4: abono no enviado a S08B1792715200”. Con esto comprobé que la devolución quedó registrada y vinculada con la transferencia compensada.

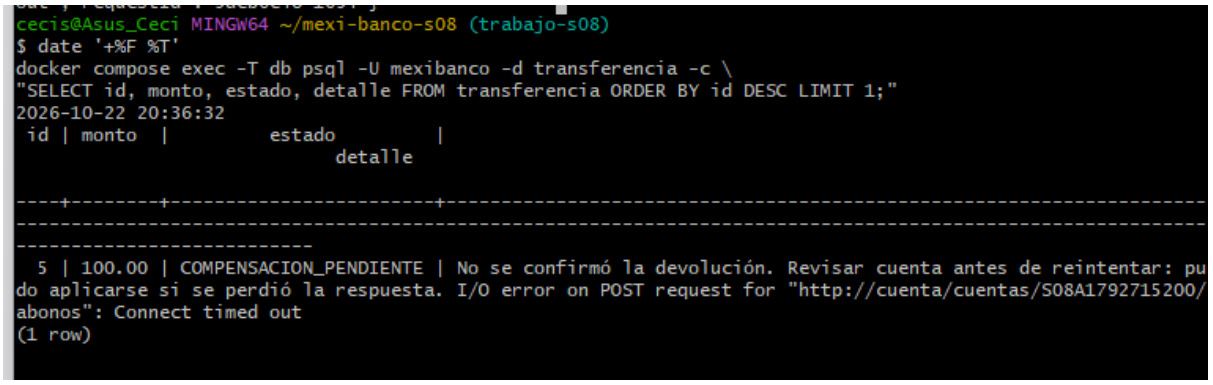


Compensación pendiente

A las 20:36:32 consulté la base de transferencia. La operación 5, por 100.00, quedó en COMPENSACION_PENDIENTE. El detalle indica que no se confirmó la devolución porque la conexión a cuenta agotó su tiempo de espera.

La operación necesita recuperación posterior. Para terminarla propondría un proceso que revise las compensaciones pendientes y reintente con espera creciente. Antes de

automatizarlo, cuenta debe reconocer un identificador único de compensación y evitar aplicar la misma devolución dos veces. Guardar el estado pendiente, por sí solo, no garantiza esa idempotencia.

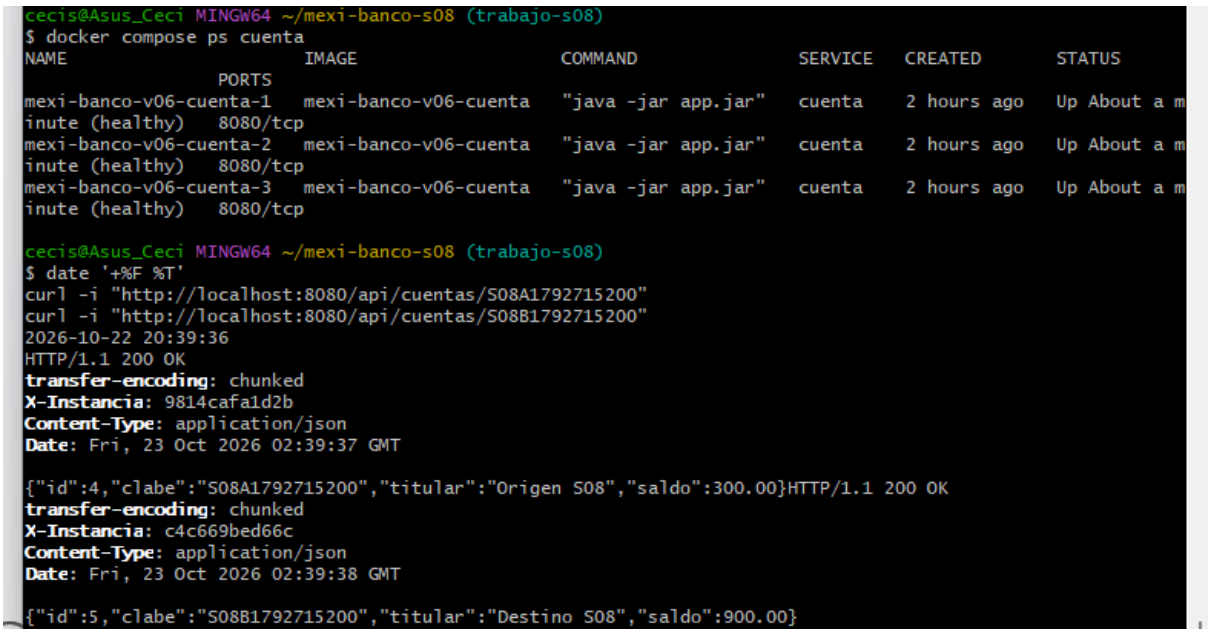




Saldos después de recuperar cuenta

A las 20:39:36 consulté las cuentas. Origen tenía 300.00 y destino 900.00. En esta prueba se descontaron 100 del origen, pero no se abonaron al destino ni se devolvieron.

Aunque cuenta volvió a estar disponible, la devolución no ocurrió automáticamente porque todavía no implementé un proceso de recuperación. La transferencia 5 permanece registrada como COMPENSACION_PENDIENTE.



Circuit breaker de cuenta

Implementé un circuit breaker manual compartido por las llamadas de CuentaCliente. Abre después de tres fallas consecutivas de disponibilidad y bloquea nuevas llamadas HTTP durante 15 segundos. Después permite una llamada de prueba en estado semiabierto: si obtiene una respuesta normal, cierra; si ocurre otra falla de disponibilidad, vuelve a abrir.

Los rechazos de negocio, como una cuenta inexistente o saldo insuficiente, no cuentan como caídas del servicio. La falla controlada del abono tampoco incrementa el contador. El circuito mantiene su estado en memoria, por lo que vuelve a cerrado cuando se reinicia transferencia.

CircuitoCuenta.java

transferencia > src > test > java > mx > mexibanco > clientes > CircuitoCuenta.java

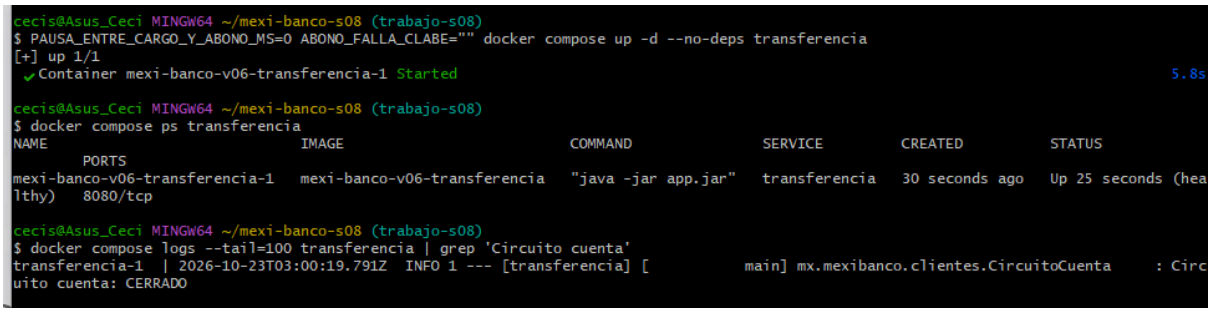
```
13      public class CircuitoCuenta {
36          public <T> T ejecutar(Supplier<T> llamada) {
51              }
52          }
53
54          private synchronized long permitirLlamada() {
55              if (estado == Estado.ABIERTO) {
56                  if (System.nanoTime() - apertura < ESPERA_NANOS) {
57                      LOG.warn("Circuito cuenta ABIERTO: llamada bloqueada sin HTTP");
58                      throw new CircuitoAbiertoException(
59                              "Cuenta temporalmente no disponible: circuito abierto");
60                  }
61
62                  cambiarEstado(Estado.SEMIABIERTO);
63                  LOG.info("Circuito cuenta: se permite una llamada de prueba");
64                  return generacion;
65              }
66
67              if (estado == Estado.SEMIABIERTO) {
68                  throw new CircuitoAbiertoException(
69                          "Cuenta en prueba de recuperación; espera antes de reintentar");
70              }
71
72              return generacion;
73          }
74
75          private synchronized void registrarExito(long permiso) {
76              if (permiso != generacion) {
77                  return;
78              }
79
80              fallas = 0;
81
82              if (estado == Estado.SEMIABIERTO) {
83                  cambiarEstado(Estado.CERRADO);
84              }
85          }
86
87          private synchronized void registrarFalla(long permiso) {
88              if (permiso != generacion) {
89                  return;
90              }
91
92              fallas++;
93              LOG.warn("Circuito cuenta: falla de disponibilidad {}", fallas);
94
95              if (estado == Estado.SEMIABIERTO || fallas >= LIMITE_FALLAS) {
96                  apertura = System.nanoTime();
97                  cambiarEstado(Estado.ABIERTO);
```

CircuitoCuenta.java | CuentaCliente.java

transferencia > src > main > java > mx > mexibanco > clientes > CuentaCliente.java

```java
15  public class CuentaCliente {
29
30      public CuentaRemota consultar(String clabe) {
31          return circuito.ejecutar(() ->
32                  http.get()
33                          .uri("/cuentas/{clabe}", clabe)
34                          .retrieve()
35                          .body(CuentaRemota.class));
36      }
37
38      public CuentaRemota cargar(String clabe, BigDecimal monto) {
39          return circuito.ejecutar(() ->
40                  http.post()
41                          .uri("/cuentas/{clabe}/cargos", clabe)
42                          .body(Map.of("monto", monto))
43                          .retrieve()
44                          .body(CuentaRemota.class));
45      }
46
47      public CuentaRemota abonarDestino(String clabe, BigDecimal monto) {
48          // La falla simulada no cuenta como caída del servicio.
49          if (!clabeConFalla.isBlank() && clabeConFalla.equals(clabe)) {
50              throw new AbonoRechazadoException(
51                      "Falla de prueba: no se envió el abono a " + clabe);
52          }
53
54          return abonar(clabe, monto);
55      }
56
57      public CuentaRemota abonar(String clabe, BigDecimal monto) {
58          return circuito.ejecutar(() ->
59                  http.post()
60                          .uri("/cuentas/{clabe}/abonos", clabe)
61                          .body(Map.of("monto", monto))
62                          .retrieve()
63                          .body(CuentaRemota.class));
64      }
65  }
```

El circuito arrancó en CERRADO. La hora corresponde a las 21:00:19 del 22 de octubre, en Ciudad de México:

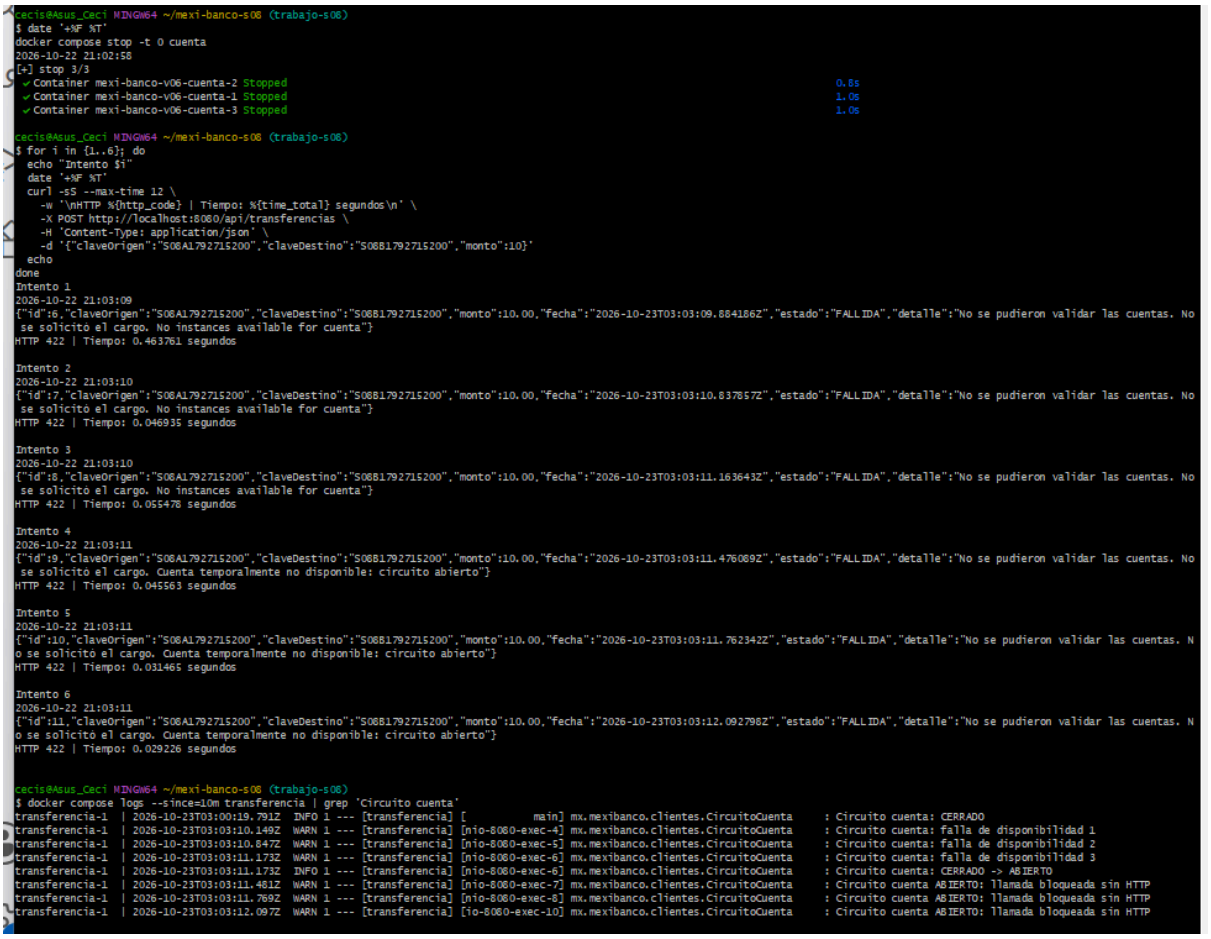


Apertura del circuit breaker

El 22 de octubre de 2026, entre las 21:03:09 y las 21:03:12, envié seis solicitudes con cuenta detenida. Las primeras tres detectaron que no había instancias disponibles. A las 21:03:11, el circuito pasó de CERRADO a ABIERTO.

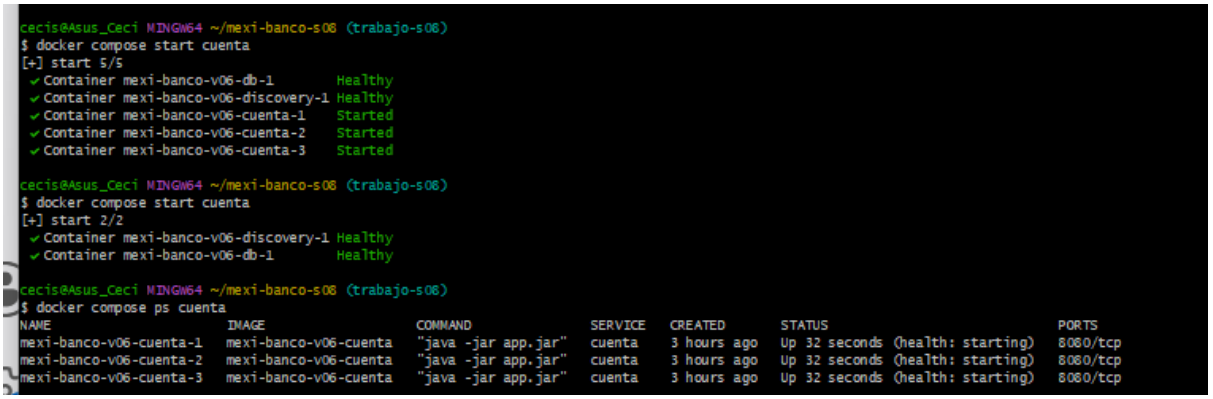
Los siguientes tres intentos fueron bloqueados antes de enviar HTTP a cuenta y respondieron en aproximadamente 0.03 a 0.05 segundos. Todas las operaciones quedaron FALLIDAS durante la validación, sin solicitar el cargo.

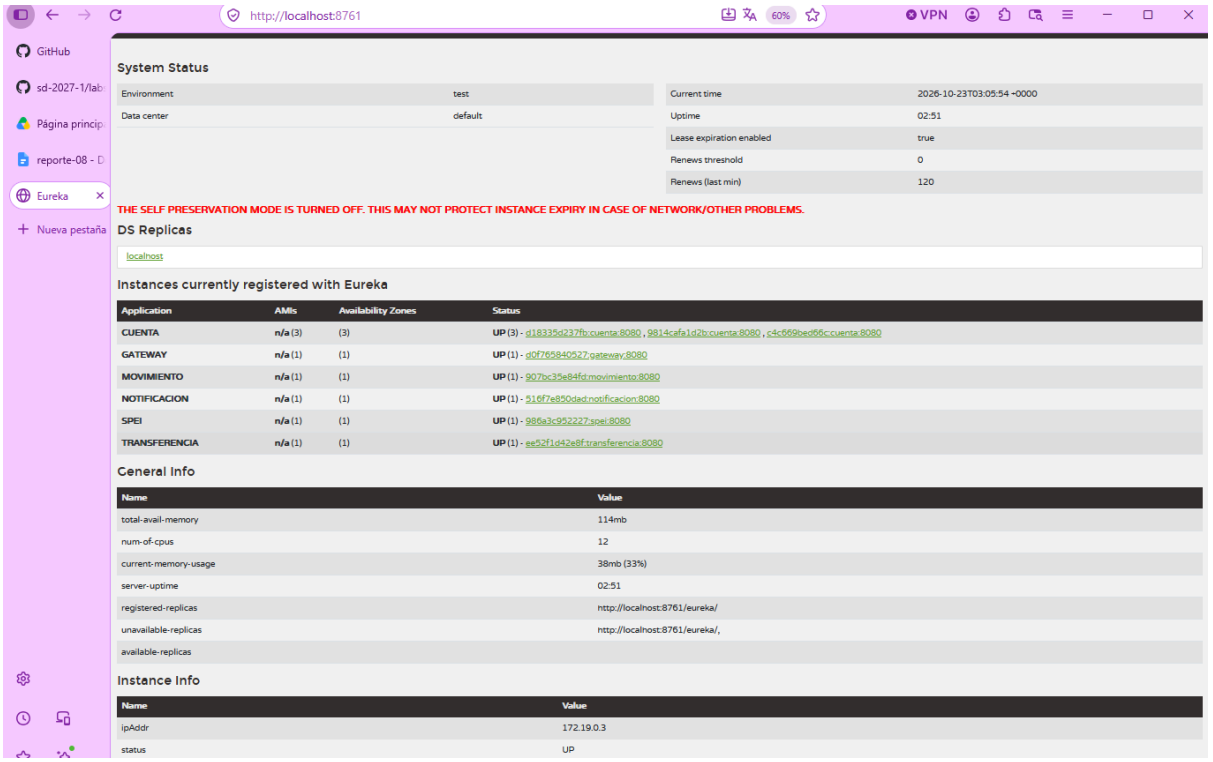
Las primeras llamadas también fueron rápidas porque el balanceador ya conocía la ausencia de instancias. Por eso, la evidencia principal del circuito son sus cambios de estado y los mensajes “llamada bloqueada sin HTTP”, no solo los tiempos.



Recuperación de cuenta

Reinicié las tres copias de cuenta y comprobé que Eureka volvía a mostrar CUENTA UP (3). No reinicié transferencia, para conservar el estado del circuit breaker y probar su recuperación con la siguiente solicitud.



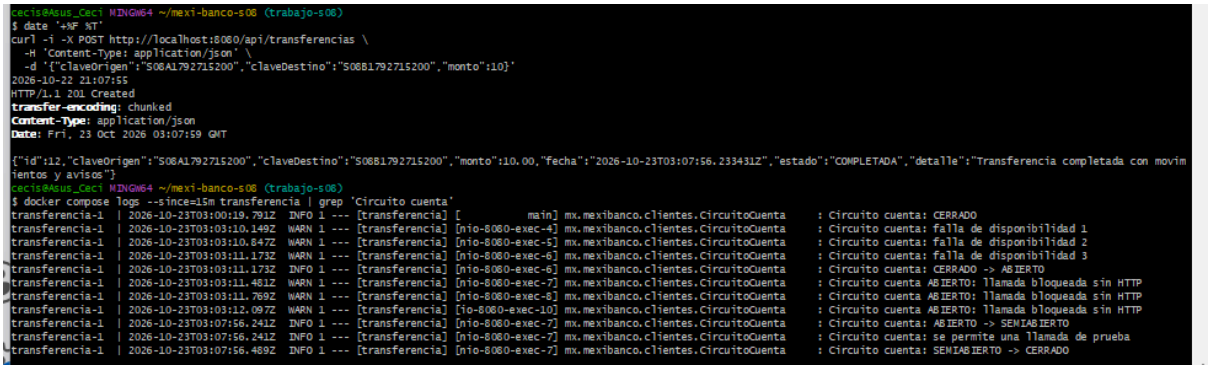


Recuperación del circuit breaker

El 22 de octubre de 2026, a las 21:07:55, solicité una transferencia de 10 después de recuperar cuenta, sin reiniciar transferencia.

A las 21:07:56, el circuito pasó de ABIERTO a SEMIABIERTO y permitió una llamada de prueba. Como la llamada funcionó, volvió a CERRADO. El cambio ocurrió al recibir la nueva solicitud, no automáticamente al encender cuenta.

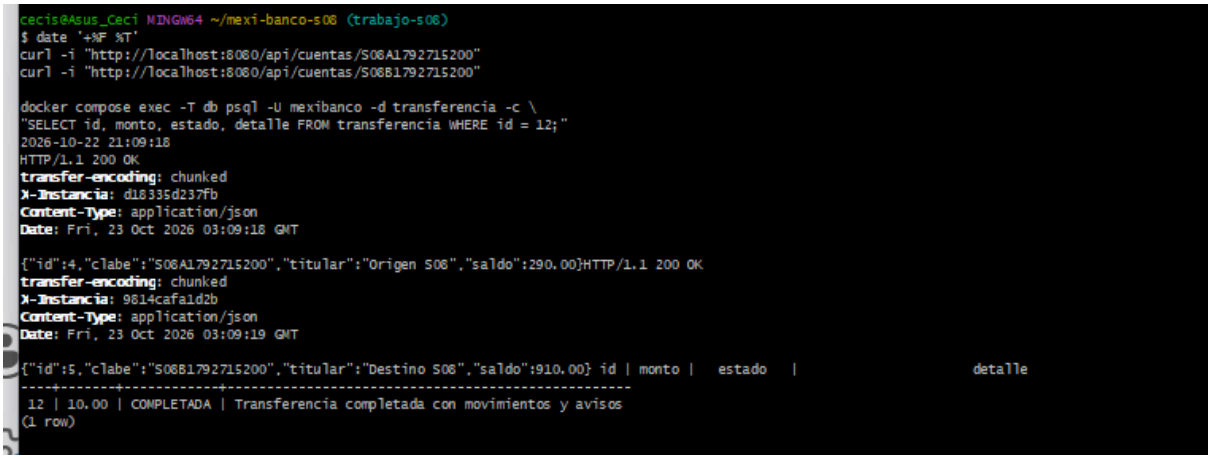
La transferencia recibió el ID 12 y respondió 201 Created, con estado COMPLETADA y el detalle de que se registraron movimientos y avisos.



Comprobación de la transferencia exitosa

A las 21:09:18 consulté las cuentas. Origen tenía 290.00 y destino 910.00, por lo que los 10 se descontaron del origen y llegaron al destino. También comprobé en PostgreSQL que la transferencia 12 quedó en COMPLETADA.

Esta operación no resolvió la compensación pendiente de la transferencia 5; únicamente comprobó que el sistema volvió a procesar nuevas transferencias después de recuperarse el circuito.

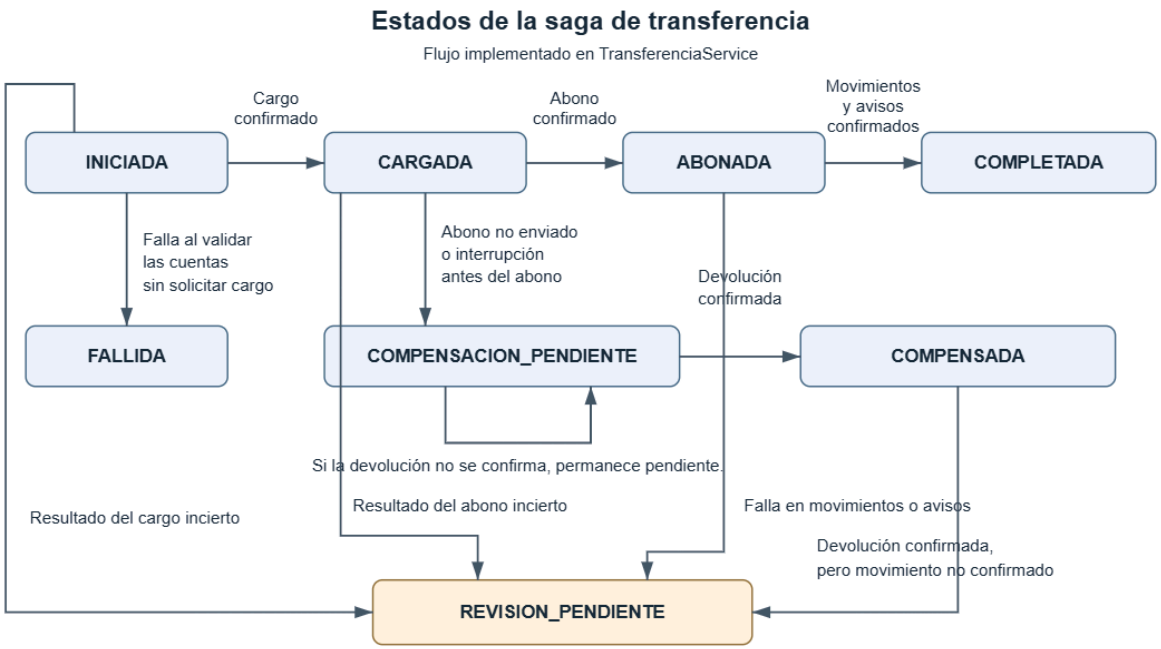


La saga no oculta los cambios intermedios. Si alguien consulta origen después del cargo y antes de la compensación, verá un saldo menor, aunque después el dinero se devuelva. Durante ese intervalo, otra operación podría encontrar saldo insuficiente. Cada paso se confirma por separado, por lo que la saga no ofrece el aislamiento de una sola transacción que abarque toda la transferencia.

El diagrama muestra los estados de una transferencia. En el camino exitoso pasa por INICIADA, CARGADA, ABONADA y COMPLETADA. Si el abono no se envía después del cargo, se intenta devolver el dinero al origen; si la devolución no se confirma, queda COMPENSACION_PENDIENTE.

COMPENSADA indica que la devolución fue confirmada. El detalle permite comprobar si también se registró su movimiento. Cuando el resultado de una llamada es incierto o falta completar un registro posterior, se utiliza REVISION_PENDIENTE para evitar repetir operaciones a ciegas.

Los estados pendientes no se recuperan automáticamente en esta implementación.

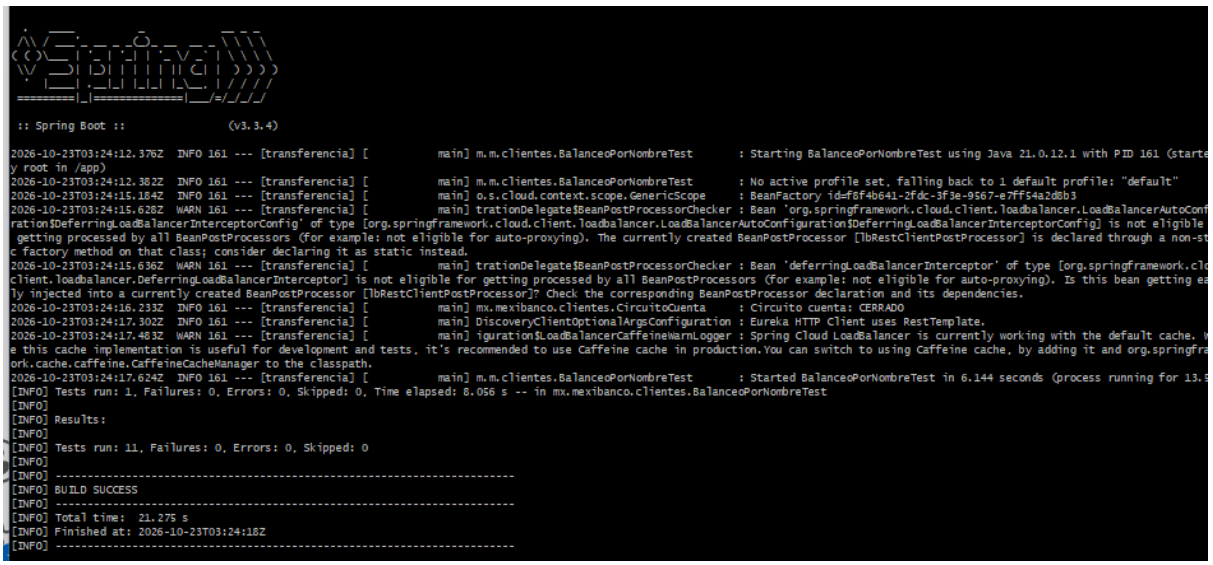


- COMPENSADA se guarda al confirmar la devolución; su detalle indica si falta el movimiento.
- Si el movimiento de devolución se confirma, permanece COMPENSADA y se actualiza el detalle.
- No hay recuperación automática. Antes de reintentar una devolución se necesita idempotencia.
- El estado registra el último paso guardado; no elimina la incertidumbre de una respuesta perdida.

Verificación final

El 22 de octubre de 2026, aproximadamente a las 21:24, ejecuté nuevamente las pruebas con la versión final del código. Se ejecutaron 11 pruebas, sin fallos, errores ni pruebas omitidas.

Estas pruebas verifican la lógica de la saga, los clientes HTTP y el balanceo por nombre. Los cambios de estado del circuit breaker se comprobaron por separado con los servicios reales y sus logs.



```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.3.4)

2026-10-23T03:24:12.376Z  INFO 161 --- [transferencia] [           main] m.m.clientes.BalanceoPorNombreTest       : Starting BalanceoPorNombreTest using Java 21.0.12.1 with PID 161 (starte
y root in /app)
2026-10-23T03:24:12.382Z  INFO 161 --- [transferencia] [           main] m.m.clientes.BalanceoPorNombreTest       : No active profile set, falling back to 1 default profile: "default"
2026-10-23T03:24:15.184Z  INFO 161 --- [transferencia] [           main] o.s.cloud.context.scope.GenericScope     : BeanFactory id=f8f4b641-2fdc-3f3e-9567-e7ff54a2d8b3
2026-10-23T03:24:15.628Z  WARN 161 --- [transferencia] [           main] trationDelegate$BeanPostProcessorChecker : Bean 'org.springframework.cloud.client.loadbalancer.LoadBalancerAutoConf
ration$DeferringLoadBalancerInterceptorConfig' of type [org.springframework.cloud.client.loadbalancer.LoadBalancerAutoConfiguration$DeferringLoadBalancerInterceptorConfig] is not eligible
 getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying). The currently created BeanPostProcessor [lbRestClientPostProcessor] is declared through a non-st
c factory method on that class; consider declaring it as static instead.
2026-10-23T03:24:15.636Z  WARN 161 --- [transferencia] [           main] trationDelegate$BeanPostProcessorChecker : Bean 'deferringLoadBalancerInterceptor' of type [org.springframework.clo
client.loadbalancer.DeferringLoadBalancerInterceptor] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying). Is this bean getting ea
ly injected into a currently created BeanPostProcessor [lbRestClientPostProcessor]? Check the corresponding BeanPostProcessor declaration and its dependencies.
2026-10-23T03:24:16.233Z  INFO 161 --- [transferencia] [           main] mx.mexibanco.clientes.CircuitoCuenta     : Circuito cuenta: CERRADO
2026-10-23T03:24:17.302Z  INFO 161 --- [transferencia] [           main] DiscoveryClientOptionalArgsConfiguration : Eureka HTTP Client uses RestTemplate.
2026-10-23T03:24:17.483Z  WARN 161 --- [transferencia] [           main] iguration$LoadBalancerCaffeineWarnLogger : Spring Cloud LoadBalancer is currently working with the default cache. W
e this cache implementation is useful for development and tests, it's recommended to use Caffeine cache in production.You can switch to using Caffeine cache, by adding it and org.springfra
ork.cache.caffeine.CaffeineCacheManager to the classpath.
2026-10-23T03:24:17.624Z  INFO 161 --- [transferencia] [           main] m.m.clientes.BalanceoPorNombreTest       : Started BalanceoPorNombreTest in 6.144 seconds (process running for 13.9
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.056 s -- in mx.mexibanco.clientes.BalanceoPorNombreTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  21.275 s
[INFO] Finished at: 2026-10-23T03:24:18Z
[INFO] ------------------------------------------------------------------------
```